

Introduction to Arrays

◆ What is an Array?

An **array** is a data structure that stores a fixed-size sequence of elements of the **same data type**. Elements are stored in **contiguous memory locations**.

Example in C:

```
int arr[5]; // declares an array of 5 integers
```

◆ Key Properties:

- **Homogeneous data:** All elements are of the same type (e.g., all int, all float).
- **Indexed:** Each element can be accessed using an index starting from 0.
- **Fixed size:** Size must be known or defined at declaration.

◆ Why Use Arrays?

- Simplifies storing multiple related values
- Allows for fast access and efficient operations using loops
- Better than declaring multiple individual variables

□ Class Activity:

- Discuss: What would be the problem if we wanted to store marks of 100 students without arrays?
- Try declaring an array and accessing elements using a loop.

📖 Declaration and Initialization

◆ Declaration Syntax:

```
data_type array_name[array_size];
```

Examples:

```
int nums[10];  
float marks[5];  
char letters[20];
```

◆ Initialization Methods:

1. Static Initialization:

```
int nums[5] = {1, 2, 3, 4, 5};
```

2. Partial Initialization

```
int nums[5] = {10, 20}; // Remaining elements will be set to 0
```

3. Runtime Initialization (Using Loops):

```
for (int i = 0; i < 5; i++)  
    scanf("%d", &nums[i]);
```

◆ Accessing Elements:

```
printf("%d", nums[2]); // prints 3rd element
```

□ Exercise:

- Declare an array and initialize it with values.
- Write a program to find the sum of array elements.

One-Dimensional Arrays

◆ What is a One-Dimensional Array?

A linear list of values stored in a single row.

Example:

```
int marks[5] = {90, 85, 78, 92, 88};
```

◆ Array Operations:

1. Traversing:

```
for(int i = 0; i < 5; i++)
    printf("%d ", marks[i]);
```

2. Input/Output:

```
for(int i = 0; i < 5; i++)
    scanf("%d", &marks[i]);
```

3. Sum and Average:

```
int sum = 0;
for(int i = 0; i < 5; i++)
    sum += marks[i];
float avg = sum / 5.0;
```

4. Maximum and Minimum:

```
int max = marks[0];
for(int i = 1; i < 5; i++)
    if(marks[i] > max) max = marks[i];
```

□ Exercise:

- Write a program to find the max, min, and average of 5 numbers.

Multi-Dimensional Arrays

◆ What is a 2D Array?

A table or matrix of rows and columns.

Syntax:

```
data_type array_name[rows][columns];
```

Example:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

◆ Accessing Elements:

```
printf("%d", matrix[1][2]); // prints 6
```

◆ Traversing 2D Array:

```
for(int i = 0; i < 2; i++)  
    for(int j = 0; j < 3; j++)  
        printf("%d ", matrix[i][j]);
```

◆ Applications:

- Matrix addition/multiplication
- Storing tabular data like student records

□ Exercise:

- Create two 2D arrays and write a program to add them.

Sorting and Searching

◆ Bubble Sort (Ascending Order):

Repeatedly compares adjacent elements and swaps them if they're in the wrong order.

```
for(i = 0; i < n-1; i++) {  
    for(j = 0; j < n-i-1; j++) {  
        if(arr[j] > arr[j+1]) {  
            // swap  
        }  
    }  
}
```

◆ Selection Sort:

Finds the smallest element and places it at the beginning.

```
for(i = 0; i < n-1; i++) {  
    min = i;  
    for(j = i+1; j < n; j++) {  
        if(arr[j] < arr[min])
```

```

        min = j;
    }
    // swap
}

```

◆ Sequential Search:

Checks each element one by one until the desired value is found.

```

for(i = 0; i < n; i++) {
    if(arr[i] == key) {
        printf("Found at index %d", i);
        break;
    }
}

```

□ Exercise:

- Implement bubble and selection sort on a list of numbers.
- Search for a specific value in an array using sequential search.

📖 String Handling

◆ What is a String?

A **string** is a 1D array of characters ending with a null character '`\0`'.

Declaration:

```
char name[10] = "John";
```

Or:

```
char name[] = {'J','o','h','n','\0'};
```

◆ Input and Output:

Program Using `gets()` and `puts()`

This program uses `gets()` to take an input string and `puts()` to print it.

```
#include <stdio.h>

int main() {
    char str[100];

    // Using gets() to take input
    printf("Enter a string using gets(): ");
    gets(str); // Input string using gets()

    // Displaying the string using puts()
    printf("\nYou entered using gets(): ");
    puts(str); // Output the string using puts()

    return 0;
}
```

Explanation:

1. **gets()**: Reads a string from the user, including spaces, and stores it in the `str` array.
2. **puts()**: Prints the string to the console with an automatic newline after it.

Program Using `scanf()` and `printf()`

This program uses `scanf()` to take a string input and `printf()` to print it.

```
#include <stdio.h>

int main() {
    char str[100];

    // Using scanf() to take input
    printf("Enter a string using scanf(): ");
    scanf("%s", str); // Input string using scanf()

    // Displaying the string using printf()
    printf("\nYou entered using scanf(): %s\n", str); // Output the string
    using printf()

    return 0;
}
```

Explanation:

1. **scanf("%s", str)**: Reads a word (without spaces) from the user and stores it in the `str` array.
2. **printf()**: Prints the string to the console.

Note:

- `gets()` is **unsafe** and should not be used in modern C programming due to potential buffer overflows. It's better to use `fgets()` in real-world programs.
- `scanf("%s", str)` reads only a single word (no spaces). If you need to read a line with spaces, you should use `fgets()` instead.

String Handling Functions:

In C, string handling functions are provided by the `<string.h>` library. Below are the explanations and examples for common string functions such as `strlen()`, `strcpy()`, `strcat()`, `strrev()`, `strupr()`, `strlwr()`, and `strcmp()`.

1. `strlen()` - Find the Length of a String

`strlen()` returns the number of characters in a string, excluding the null terminator (`\0`).

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, World!";
    printf("Length of the string: %d\n", strlen(str));
    return 0;
}
```

Output:

Length of the string: 13

2. `strcpy()` - Copy One String to Another

`strcpy()` copies the content of one string (source) to another string (destination).

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello, World!";
    char dest[50];
    strcpy(dest, src);
    printf("Source: %s\n", src);
    printf("Destination: %s\n", dest);
}
```

```
    return 0;
}
```

Output:

Source: Hello, World!
Destination: Hello, World!

3. `strcat()` - Concatenate Two Strings

`strcat()` concatenates (appends) the source string to the destination string.

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello, ";
    char str2[] = "World!";
    strcat(str1, str2); // Concatenate str2 to str1
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

Output:

Concatenated string: Hello, World!

4. `strrev()` - Reverse a String

`strrev()` reverses the given string in place.

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello";
    strrev(str);
    printf("Reversed string: %s\n", str);
    return 0;
}
```

Output (if `strrev()` is available):

Reversed string: olleH

5. **strupr()** - Convert String to Uppercase

`strupr()` converts all characters of the string to uppercase.

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "hello";
    strupr(str);
    printf("Uppercase string: %s\n", str);
    return 0;
}
```

Output :

Uppercase string: HELLO

6. **strlwr()** - Convert String to Lowercase

`strlwr()` converts all characters of the string to lowercase.

Syntax:

```
char *strlwr(char *str);
```

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "HELLO";
    strlwr(str);
    printf("Lowercase string: %s\n", str);
    return 0;
}
```

Output :

Lowercase string: hello

7. **strcmp()** - Compare Two Strings

`strcmp()` compares two strings lexicographically. It returns:

- **0** if the strings are equal.

- A **positive value** if the first string is lexicographically greater than the second.
- A **negative value** if the first string is lexicographically smaller than the second.

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "Hello";
    char str2[] = "hello";

    int result = strcmp(str1, str2);

    if(result == 0) {
        printf("Strings are equal.\n");
    } else if(result > 0) {
        printf("'%s' is greater than '%s'.\n", str1, str2);
    } else {
        printf("'%s' is less than '%s'.\n", str1, str2);
    }
    return 0;
}
```

Output:

'Hello' is less than 'hello'.

- ASCII value of H: 72
- ASCII value of h: 104

Summary of Functions:

Function	Description
strlen()	Returns the length of a string (excluding null terminator).
strcpy()	Copies the source string to the destination string.
strcat()	Concatenates the source string to the destination string.
strrev()	Reverses the string (non-standard in C).
strupr()	Converts a string to uppercase (non-standard in C).
strlwr()	Converts a string to lowercase (non-standard in C).
strcmp()	Compares two strings lexicographically.

WAP to ask 5 marks and find its sum and average.

WAP to ask 5 salary and display salary between 15000 and 25000.

WAP to find largest and smallest element in an array.

WAP to display selection sort.

WAP to display bubble sort.

WAP to search an element using sequential search.

WAP to find sum of two MXN matrix.

WAP to find transpose of two MXN matrix.

WAP to find product of two MXN matrix.

WAP to read a string and count the number of vowels, consonants, space, semicolons and commas in the string.

WAP to read name list of 5 student and display in alphabetic form.

WAP to check whether a no is palindrome or not.